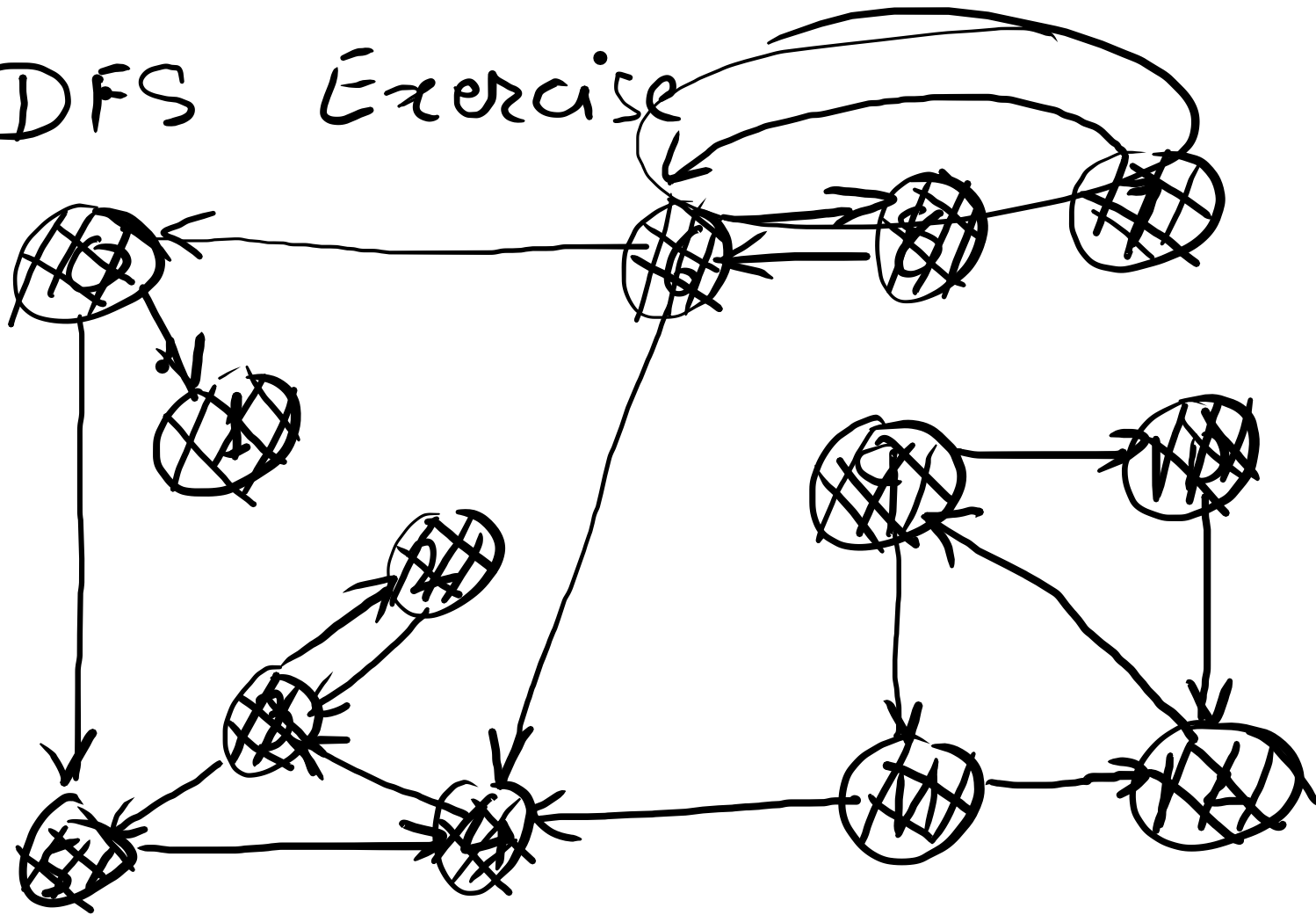


DFS Exercise



7
1
6
0
1
5

2
3
4
5
1
0

4
9
2
10

8
6

7

T1	T2	T3	T4
0	10	6	7
1	1	1	
5	12	8	
4	1	7	
3	9	1	
1	11	6	
2		8	

Forward: (0, 5), (0, 1)
 (5, 4) (4, 3) (3, 2) (6, 8)
 (10, 12) (12, 9) (9, 11)
 Backward: (3, 5) (2, 3)
 (8, 6) (9, 10)
 cross: (6, 0) (6, 4) (7, 6)

Applications of DFS:

- ① Program control flow analysis
- ② Garbage collection
- ③ Cycle detection
- ④ Topological sort

Cycle Detection using DFS

$v_1 \rightarrow v_2 \rightarrow \dots \rightarrow v_\ell$ and $v_1 = v_\ell$

$\therefore$ there is an edge $v_{\ell-1} \rightarrow v_1$
backward edge

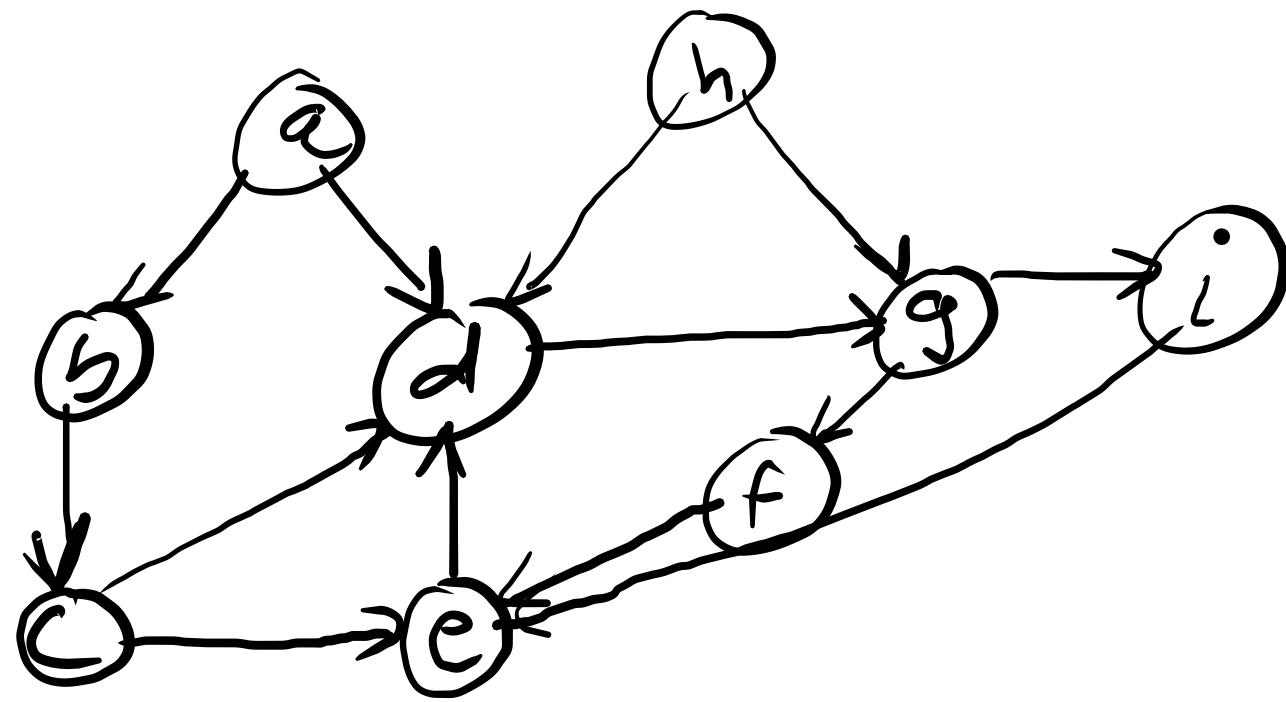
— Suppose DFS already executed and we have

DFS Forest T

- Let (u, v) be an edge in G . We classify edge into
 - Forward edge: u is proper ancestor of v
 - Backward edge: u is proper descendent of v
 - Cross edge: neither of the above is true

Edge classification:

- Determine the type of edge (u, v) in constant time
- Requires to augment DFS algorithm by remembering when each vertex enters and leaves stack
- Augmenting DFS algorithm:
 - keep counter $c = 0$
 - Every time we push or pop vertex in or out of stack increment c ($c = c + 1$)
 - For every vertex v define:
 - Discovery time $d\text{-}tm(v)$ be the value of c after v is pushed into stack
 - Finish time $f\text{-}tm(v)$ be the value of c after v is popped from stack
 - $l(v) = [d\text{-}tm(v), f\text{-}tm(v)]$



$$l(d) = [1, 10]$$

$$l(g) = [2, 9]$$

$$l(f) = [3, 6]$$

$$l(e) = [4, 5]$$

$$l(i) = [7, 8]$$

